Autonomous Duck Deployment Framework

> *A file-first methodology for coordinating LLM-assisted research, design, development, review, handoff, and long-term project evolution.*

Framework version: 3.5
Short name: ADDF
Primary principle: The files are the project.
Operating modes: Research Mode, Design Mode, Develop Mode
Work scale: Project → Release → Feature → Sprint → Patch
Lifecycle: Research → Design & feasibility → Validation → Architecture → Sprint planning → Build & test → Review & reflection → Deploy, maintain, or resume

---

Contents

1. Purpose

ADDF is a coordination framework for LLM-assisted software delivery.

It gives engineers a file-backed method for moving from research to design, from design to implementation, from implementation to review, and from review to long-term maintenance. The framework does not depend on a specific model, editor, agent, repository host, language, or deployment platform.

ADDF assumes the model is temporary compute. The repository is the durable system of record.

The framework defines:

how project memory is represented,

how LLM sessions are constrained,

how work is scoped,

how implementation is approved,

how releases and features evolve,

how one model hands work to another,

and how a project resumes after context loss.

Rule: The model assists the project. It does not become the project.

2. The problem ADDF solves

Unstructured AI-assisted development fails when the chat thread becomes the project memory.

A chat can hold conversation. It cannot reliably hold long-term system truth.

2.1 Chat-thread memory does not scale

A long AI thread accumulates stale code blocks, superseded requirements, abandoned fixes, environment assumptions, and unrelated discussion. The model may continue to treat that history as relevant long after the project has changed.

The result is familiar:

old decisions reappear,

fixed bugs return,

file names drift,

scope expands without approval,

and the developer spends more time correcting the model than directing the work.

ADDF moves project memory into files.

2.2 Plausibility is not correctness

When a specification has gaps, the model fills them.

That behavior produces plausible output. It does not guarantee correct output.

Typical failures include:

invented schema fields,

invented API routes,

unapproved dependencies,

hidden refactors,

permission assumptions,

inconsistent naming,

test summaries without real verification,

and business logic that never appeared in the requirements.

ADDF reduces these failures by making the domain, state, decisions, requirements, blueprint, acceptance criteria, dry run, and implementation log explicit.

2.3 Context size is not context quality

Large context windows help. They do not replace context discipline.

A large context window filled with stale, contradictory, or irrelevant material makes the model less reliable. ADDF prioritizes curated context over bulk context.

Load the files required for the current mode. Leave the rest out.

Rule: Cleaner context beats more context.

3. Framework summary

[Image placeholder: 1. ADDF Framework Map goes here]

ADDF separates four decisions that AI-assisted workflows often collapse into one.

```
1. Work scale — How large is the change?
2. Lifecycle stage — Where is the work in the delivery process?
3. Operating mode — What is the model allowed to do?
4. Project memory — Which files define the current truth?
```

The scale model is:

```
Project → Release → Feature → Sprint → Patch
```

The lifecycle is:

```
Research
→ Design & feasibility
→ Validation
→ Architecture
→ Sprint planning
→ Build & test
→ Review & reflection
→ Deploy, maintain, or resume
```

The operating modes are:

```
Research Mode
Design Mode
Develop Mode
```

The project memory layer is the project brain:

```
AGENTS.md
DOMAIN.md
STATE.md
DECISIONS.md
COMMANDS.md
SECURITY.md
QUESTIONS.md
RISKS.md
requirements.md
blueprint.md
acceptance.md
dry_run.md
implementation_log.md
```

```
human_review.md
retrospective.md
```

ADDF is recursive. The full lifecycle applies to a project, a major release, a high-risk feature, or a pivot. Smaller work uses a smaller slice of the lifecycle.

Rule: Choose the scale before choosing the process.

---

4. Core principles

4.1 Local File Sovereignty

The authoritative project state lives in local files.

LLM sessions can read, summarize, modify, or produce those files according to mode constraints. The model is not the system of record. The repository is.

Canonical project memory belongs in Markdown files that can be reviewed, diffed, committed, searched, and loaded into any capable LLM session.

Rule: The files are the project.

4.2 Explicit work scale

Every unit of work receives a scale before execution.

```
Project
Release
Feature
Sprint
Patch
```

Scale determines process depth. A project requires research, design, validation, architecture, implementation, review, and release planning. A patch may require one edit and one commit.

Rule: Do not run a project process for a patch. Do not run a patch process for a project.

4.3 Mode-constrained LLM sessions

Every AI session runs in one explicit mode.

```
Research Mode
Design Mode
Develop Mode
```

Each mode has different write permissions.

Research Mode investigates. Design Mode writes project memory. Develop Mode changes implementation.

Rule: Declare the mode before the model acts.

4.4 Dry run before mutation

Develop Mode begins with a dry run.

The model lists intended file changes, commands, dependency requests, risks, and assumptions. Then it stops. Implementation begins only after review and authorization.

The dry run creates the review boundary that prevents hidden scope creep.

Rule: No dry run, no implementation.

4.5 Reviewable project memory

Significant work produces artifacts that survive the session.

Examples:

```
requirements.md
blueprint.md
acceptance.md
dry_run.md
implementation_log.md
human_review.md
retrospective.md
DECISIONS.md
STATE.md
```

These artifacts let the human inspect scope, verify output, recover from failure, and hand work to another model.

Rule: If the work matters, it leaves a file trail.

4.6 Handoff by file, not by chat

Model handoff uses files, not prior conversation.

If Codex reaches a context limit and Claude takes over, Claude receives the project files, sprint files, implementation log, and handoff summary. The previous chat is not required.

Rule: A valid handoff survives without the old thread.

5. The three operating modes

[Image placeholder: 2. Three Operating Modes diagram goes here]

ADDF has three operating modes.

```
Research Mode
Design Mode
Develop Mode
```

The mode answers one question:

```
What is the model allowed to do in this session?
```

5.1 Research Mode

Research Mode investigates and synthesizes information.

Use Research Mode when the team needs to understand a domain, compare tools, summarize documentation, evaluate constraints, inspect prior art, identify risks, or list unknowns.

Research Mode may produce:

```
research_notes.md
source_summary.md
tool_comparison.md
open_questions.md
risk_notes.md
research_summary.md
```

Research Mode must not:

write source code,

mutate implementation files,

make final architecture decisions,

add dependencies,

or convert uncertain findings into binding project rules.

Rule: Research Mode gathers evidence. It does not decide or build.

5.2 Design Mode

Design Mode writes the project memory.

Use Design Mode to convert research, intent, requirements, feedback, implementation logs, and decisions into durable Markdown artifacts.

Design Mode covers:

feasibility analysis,
MVP definition,
validation gates,
architecture planning,
release planning,
feature planning,
sprint pack generation,
dry run review,
implementation review,
retrospective generation,
resumption summaries,
handoff notes,
documentation updates,
and decision logging.

Design Mode may create or update:

```
DOMAIN.md
STATE.md
DECISIONS.md
COMMANDS.md
STYLE_GUIDE.md
SECURITY.md
QUESTIONS.md
RISKS.md
GIT_STRATEGY.md
PROMPT_CHANGELOG.md
research_notes.md
design_options.md
feasibility.md
validation.md
release_plan.md
feature_brief.md
requirements.md
```

```
blueprint.md
acceptance.md
dry_run_review.md
human_review.md
retrospective.md
handoff_summary.md
planning/backlog.md
docs/architecture.md
docs/data_model.md
docs/api.md
```

Design Mode must not:

write source code,

mutate implementation files,

add dependencies directly,

run uncontrolled repository changes,

or treat a plan as implementation.

Rule: Design Mode writes Markdown project memory. It does not write implementation.

5.3 Develop Mode

Develop Mode changes the actual project.

It is the only mode allowed to write code or modify implementation assets.

Develop Mode may create or modify:

```
src/
app/
components/
tests/
scripts/
website files
game project files
config files
package files
build files
```

```
tooling files
approved assets
```

Develop Mode may also update implementation-related Markdown:

```
dry_run.md
implementation_log.md
handoff_summary.md
```

Develop Mode must:

start at Permission Level 0,
produce a dry run,
stop after the dry run,
wait for explicit approval,
modify only approved files,
log every change,
run verification commands,
paste raw output into `implementation_log.md`,
stop when ambiguity appears,
and checkpoint before context limits or model handoff.

Develop Mode must not:

invent business rules,
rewrite planning files unless explicitly authorized,
add dependencies without a recorded decision,
touch unrelated files,
skip dry run,
skip verification,
or continue after failures without logging them.

Rule: Develop Mode changes implementation files only after dry run approval.

5.4 Mode boundaries

MODE	PRIMARY ACTION	WRITES	MUST NOT WRITE
Research	Investigate	Research notes, summaries, open questions	Code, final decisions
Design	Specify	Markdown project memory	Source code, implementation files
Develop	Implement	Approved code, tests, implementation logs	Unapproved files, invented rules

A lifecycle step may involve more than one mode. A session has one explicit mode.

Rule: Mode controls write permission.

6. Work scale model

[Image placeholder: 3. Work Scale Model diagram goes here]

The Work Scale Model answers:

```
How large is this unit of work?
```

Scale controls the process depth.

6.1 Project

A Project is the full product or initiative.

Examples:

```
ADDF public website and starter kit
Mini Task Tracker
Unity training simulation
Local politics education site
Data ingestion pipeline
```

Project-level work generally uses the full lifecycle.

6.2 Release

A Release is a version of a project.

Examples:

```
v0.1 Public Proof
v0.2 Website MVP
v0.3 CLI Init Tool
v1.0 Stable Release
v2.0 Major Update
```

A release groups features into a bounded delivery target.

6.3 Feature

A Feature is a user-visible or system-visible capability.

Examples:

```
Starter kit download
Prompt catalog page
CLI init command
Web onboarding app
Search
```

```
CSV import
Enemy AI behavior
```

A feature may require research, design, validation, and one or more sprints.

6.4 Sprint

A Sprint is a bounded implementation packet.

In ADDF, a sprint is defined by scope and artifacts, not duration.

A sprint typically contains:

```
requirements.md
blueprint.md
acceptance.md
dry_run.md
implementation_log.md
human_review.md
retrospective.md
rollback_log.md
```

6.5 Patch

A Patch is a small, low-risk change.

Examples:

```
Fix a typo
Update a broken link
Rename one heading
Adjust one CSS token
Correct one command
```

A patch does not require the full sprint loop unless it touches logic, dependencies, architecture, data, or multiple files.

6.6 Scale-to-process mapping

WORK SCALE	PROCESS DEPTH	TYPICAL MODES
Project	Full lifecycle	Research, Design, Develop
Release	Full or partial lifecycle	Research, Design, Develop
Feature	Feature cycle	Research if needed, Design, Develop
Sprint	Sprint loop	Design, Develop
Patch	Minimal path	Human-only or Develop, Design if state changes

Rule: Process depth follows risk and scope.

7. The 8-step ADDF lifecycle

[Image placeholder: 5. 8-Step ADDF Lifecycle diagram goes here]

The lifecycle describes the delivery path.

It is scalable. Apply it deeply to a new project or major release. Apply it lightly to a bounded feature. Skip irrelevant stages for low-risk patches.

7.1 Research

Primary mode: Research Mode
Purpose: Understand the problem space before committing to a direction.

Inputs may include:

```
raw notes
business context
user needs
external documentation
technical constraints
unknowns
```

Outputs may include:

```
research_notes.md
source_summary.md
tool_comparison.md
open_questions.md
risk_notes.md
```

Exit criteria:

meaningful unknowns are documented,

relevant sources are summarized,

risks are visible,

and the work is ready for design.

Rule: Do not design against unknowns you have not named.

7.2 Design & feasibility

Primary mode: Design Mode
Purpose: Convert research into viable approaches and evaluate constraints.

Outputs may include:

```
design_options.md
feasibility.md
mvp_scope.md
```

```
risk_matrix.md
prototype_plan.md
```

Exit criteria:

at least one viable approach is documented,

MVP boundaries are clear,

major tradeoffs are recorded,

and high-risk assumptions are identified.

Rule: Feasibility work narrows options before architecture hardens them.

7.3 Validation gate

Primary mode: Design Mode + human review
Purpose: Decide whether the work is defined enough to proceed.

The validation gate asks:

Is the goal clear?

Is the target user clear?

Is scope bounded?

Are security boundaries known?

Are high-risk assumptions visible?

Are open questions acceptable or resolved?

Is the preferred approach explicit?

Outputs may include:

```
validation.md
approved_approach.md
updated QUESTIONS.md
updated RISKS.md
```

Exit criteria:

human approval exists,

blockers are resolved or explicitly deferred,

and the work is ready for architecture or sprint planning.

Rule: No validation, no architecture.

7.4 Architecture

Primary mode: Design Mode
Purpose: Convert validated intent into durable project memory.

Outputs may include:

```
DOMAIN.md
STATE.md
DECISIONS.md
COMMANDS.md
STYLE_GUIDE.md
SECURITY.md
docs/architecture.md
docs/data_model.md
docs/api.md
QUESTIONS.md
RISKS.md
```

Exit criteria:

domain rules are explicit,

command paths are documented,

security boundaries are known,

major decisions are recorded,

and current state is accurate.

Rule: Architecture lives in files before Develop Mode starts.

7.5 Sprint planning

Primary mode: Design Mode
Purpose: Convert architecture or feature intent into an implementation-ready sprint pack.

Outputs:

```
requirements.md
blueprint.md
acceptance.md
```

Exit criteria:

requirements are specific,

blueprint lists files and implementation steps,

assumptions are explicit,

acceptance criteria are checkable,

and the human has reviewed scope.

Rule: The sprint pack is the contract for Develop Mode.

7.6 Build & test

Primary mode: Develop Mode
Purpose: Implement approved work under controlled permissions.

Develop Mode first produces:

```
dry_run.md
```

Then stops.

After approval, Develop Mode implements approved changes and produces:

```
source files
tests
implementation_log.md
```

```
test output
handoff notes
```

Exit criteria:

implementation touched only approved files,

verification ran,

raw output is logged,

failures are documented,

and no unapproved dependencies were added.

Rule: Develop Mode does not self-authorize.

7.7 Review & reflection

Primary mode: Design Mode + human review
Purpose: Verify output, capture lessons, and update project memory.

Outputs may include:

```
human_review.md
retrospective.md
updated STATE.md
updated DECISIONS.md
updated planning/backlog.md
updated AGENTS.md
```

Exit criteria:

work is approved or rejected,

known issues are captured,

decisions are recorded,

state reflects reality,

and the next action is clear.

Rule: A sprint is not complete until the project brain is current.

7.8 Deploy, maintain, or resume

Primary mode: Design Mode and/or Develop Mode
Purpose: Decide the next operational state after review.

The project may:

deploy,

start another sprint,

start a new feature cycle,

start a new release cycle,

enter maintenance,

pause,

resume later,

or pivot.

Outputs may include:

```
release_notes.md
rollback_log.md
handoff_summary.md
updated COMMANDS.md
updated RISKS.md
updated STATE.md
```

Exit criteria:

deployment or distribution steps are documented,

rollback exists when needed,

state is current,

and handoff notes exist if pausing.

Rule: A project can pause only after state is written down.

8. How scale, lifecycle, and modes interlock

[Image placeholder: 4. Scale / Lifecycle / Mode Interlock diagram goes here]

ADDF separates three decisions.

8.1 Scale

```
How large is this work?
```

Answers:

```
Project / Release / Feature / Sprint / Patch
```

8.2 Lifecycle stage

```
Where is the work in the delivery process?
```

Answers:

```
Research / Design & feasibility / Validation / Architecture / Sprint planning / Build & test / Re
```

8.3 Mode

```
What is the model allowed to do?
```

Answers:

```
Research / Design / Develop
```

8.4 Mapping

LIFECYCLE STEP	PRIMARY MODE
Research	Research
Design & feasibility	Design
Validation gate	Design + human
Architecture	Design
Sprint planning	Design
Build & test	Develop
Review & reflection	Design + human
Deploy, maintain, or resume	Design and/or Develop

8.5 Examples

New project

```
Scale: Project
Lifecycle: Full 8 steps
Modes: Research → Design → Develop → Design review
```

New feature

```
Scale: Feature
Lifecycle: Feature-sized path
Modes: Research if needed → Design → Develop → Design review
```

Sprint

```
Scale: Sprint
Lifecycle: Sprint planning → Build & test → Review
Modes: Design → Develop → Design
```

Patch

```
Scale: Patch
Lifecycle: Minimal
Modes: Human-only or Develop, with Design only if state or docs change
```

Rule: Scale chooses the process depth. Lifecycle describes stage. Mode controls model permissions.

---

9. The project brain

[Image placeholder: 6. Project Brain File Map goes here]

The project brain is the file-backed memory layer.

It must be current, explicit, scoped, reviewable, searchable, versioned, and safe to load into an LLM session.

9.1 Core files

```
AGENTS.md
DOMAIN.md
STATE.md
DECISIONS.md
COMMANDS.md
STYLE_GUIDE.md
SECURITY.md
QUESTIONS.md
RISKS.md
GIT_STRATEGY.md
PROMPT_CHANGELOG.md
```

9.2 Sprint files

```
requirements.md
blueprint.md
acceptance.md
dry_run.md
implementation_log.md
human_review.md
retrospective.md
rollback_log.md
```

9.3 Release and feature files

```
release_plan.md
scope.md
release_notes.md
feature_brief.md
feasibility.md
validation.md
research.md
sprint_map.md
```

9.4 File responsibilities

FILE	RESPONSIBILITY
`AGENTS.md`	Mode rules, permissions, session constraints.
`DOMAIN.md`	Business logic, entities, workflows, boundaries.
`STATE.md`	Current goal, scale, mode, release, sprint, blockers, next action.
`DECISIONS.md`	Durable architecture and dependency decisions.
`COMMANDS.md`	Run, test, build, deploy, and rollback commands.
`SECURITY.md`	Safe-to-load and never-load rules.
`QUESTIONS.md`	Unknowns and blockers.
`RISKS.md`	Known risks and mitigation.
`requirements.md`	What the sprint must accomplish.
`blueprint.md`	How the sprint will be implemented.
`acceptance.md`	What counts as done.
`dry_run.md`	What Develop Mode intends to change.
`implementation_log.md`	What Develop Mode changed and verified.
`human_review.md`	Human approval record.
`retrospective.md`	Lessons and constraints from the sprint.

Rule: If a model needs to know it later, write it to the project brain.

10. Release cycles

ADDF supports versioned delivery.

A completed release becomes the baseline for the next release. The project brain persists across versions.

10.1 Release plan

A release plan defines:

release goal,

in-scope features,

out-of-scope items,

required research,

risks,

sprint sequence,

release criteria,

release notes,

and retrospective criteria.

10.2 Release folder structure

```
planning/
├── backlog.md
└── releases/
    ├── v0.1/
    │   ├── release_plan.md
    │   ├── scope.md
    │   ├── release_notes.md
    │   ├── retrospective.md
    │   └── sprints/
    │       ├── sprint_001/
    │       └── sprint_002/
    └── v0.2/
        ├── release_plan.md
```

```
        └── sprints/
            └── sprint_001/
```

10.3 Starting a new version

To begin V2 after V1, load:

```
DOMAIN.md
STATE.md
DECISIONS.md
RISKS.md
QUESTIONS.md
planning/backlog.md
planning/releases/v1.0/retrospective.md
user feedback
open issues
```

Use Design Mode to generate:

```
planning/releases/v2.0/release_plan.md
planning/releases/v2.0/scope.md
planning/releases/v2.0/research_questions.md
updated STATE.md
updated planning/backlog.md
updated RISKS.md
```

10.4 Release completion criteria

A release is complete when:

in-scope features are complete or intentionally deferred,

release criteria are met,

docs are updated,

changelog is updated,

release notes exist,

known issues are in the backlog,

deployment or distribution is complete,

and release retrospective exists.

Rule: A new release updates the project brain. It does not restart the project from zero.

---

11. Feature cycles

A feature cycle adds a capability to an existing project.

It does not restart the full project lifecycle unless the feature is foundational, high-risk, or direction-changing.

11.1 Feature flow

```
1. Add idea to backlog
2. Write feature brief
3. Research if needed
4. Design feasibility if needed
5. Validate scope
6. Generate sprint pack
7. Develop with dry run
8. Review and update project memory
```

11.2 Feature folder structure

```
planning/
└── features/
    └── [feature-name]/
        ├── feature_brief.md
```

```
        ├── research.md
        ├── feasibility.md
        ├── validation.md
        ├── decisions.md
        └── sprint_map.md
```

11.3 Feature brief template

```
# Feature Brief: [Feature Name]

## Summary
[What capability is being added?]

## User Value
[Why does this matter?]

## In Scope
- [Item]

## Out of Scope
- [Item]

## Known Constraints
- [Constraint]

## Research Needed
- [Question or "None"]

## Feasibility Risks
- [Risk or "None"]

## Acceptance Summary
- [High-level done condition]

## Related Files
- [Relevant files or folders]

## Release Target
[Version]
```

Rule: A feature cycle extends the project. It does not erase existing state.

12. Sprint loop

[Image placeholder: 7. Sprint Loop diagram goes here]

The Sprint Loop is the controlled implementation workflow.

Use it for bounded work that requires a plan, dry run, implementation, verification, and review.

12.1 Overview

```
1. Pre-flight gate
2. Design sprint pack
3. Human blueprint review
4. Develop dry run
5. Design review of dry run
6. Dependency approval gate
7. Authorize development
8. Build & test
9. Design review of output
10. Human approval gate
11. Commit, update, reset
```

12.2 Pre-flight gate

Check:

scale,

blockers,

risks,

branch,

relevant release or feature,

and scope clarity.

12.3 Design sprint pack

Design Mode creates:

```
requirements.md
blueprint.md
acceptance.md
```

12.4 Human blueprint review

The human validates:

desired outcome,

domain alignment,

file scope,

assumptions,

testability,

and out-of-scope boundaries.

12.5 Develop dry run

[Image placeholder: 8. Dry Run Approval Gate diagram goes here]

Develop Mode starts at Level 0 and produces:

```
dry_run.md
```

The dry run lists intended changes, commands, dependencies, risks, and assumptions.

12.6 Design review of dry run

Design Mode compares:

```
requirements.md
blueprint.md
acceptance.md
dry_run.md
```

It returns approval, required corrections, or revised permission level.

12.7 Dependency approval gate

Any new dependency requires a `DECISIONS.md` entry before implementation.

12.8 Authorize development

The human authorizes Develop Mode explicitly.

```
Dry run approved.
Permission Level [LEVEL] authorized.
Proceed according to requirements.md, blueprint.md, acceptance.md, and dry_run.md.
```

12.9 Build & test

Develop Mode implements approved work and logs:

```
implementation_log.md
test output
errors
handoff notes
```

12.10 Design review of output

Design Mode reviews implementation against plan, acceptance criteria, logs, and changed files.

12.11 Human approval gate

The human checks scope, verification, documentation, security, state, decisions, and handoff readiness.

12.12 Commit, update, reset

The human commits the work, updates project brain files, and closes or checkpoints the active AI session.

Rule: A sprint ends when state is updated, not when code compiles.

13. Develop Mode permission levels

Permission levels apply only to Develop Mode.

LEVEL	NAME	MEANING
0	Dry Run Only	Read-only analysis of intended changes. No implementation.
1	Approved Sprint Scope	Modify only files approved in `blueprint.md` and `dry_run.md`.
2	Approved Expansion	Add minor helper files surfaced in the dry run and explicitly approved.

LEVEL	NAME	MEANING
3	Supervised Refactor	Modify adjacent files with explicit approval and detailed logging.
4	Migration / High-Risk Change	Structural changes, migrations, or architecture-heavy work. Requires rollback plan.

Most work starts at Level 0 and proceeds to Level 1.

Levels 2–4 require stronger justification and review.

Rule: Permission level controls Develop Mode freedom.

14. Handoff, resumption, and model switching

[Image placeholder: 9. Handoff and Resumption diagram goes here]

ADDF treats model handoff as a normal operating condition.

14.1 Session checkpoint protocol

Before ending a session, switching models, or approaching context limits, the active AI must update or produce:

```
STATE.md
implementation_log.md if in Develop Mode
QUESTIONS.md
RISKS.md
planning/backlog.md
```

```
retrospective.md if appropriate
handoff_summary.md
```

14.2 Handoff summary

```
# Handoff Summary

## Current Goal
[Current objective]

## Current Mode
[Research / Design / Develop]

## Active Scale
[Project / Release / Feature / Sprint / Patch]

## Active Release
[Version or "None"]

## Active Feature
[Feature or "None"]

## Active Sprint
[Sprint or "None"]

## Completed Work
[Completed items]

## Files Changed
- [File path]: [Change summary]

## Commands Run
- [Command]: [Result]

## Current Problems
[Errors, blockers, failed checks]

## Unfinished Work
[Remaining work]

## Next Recommended Action
```

```
[Next safest action]

## Warnings
[Risks, assumptions, caveats]
```

14.3 Incoming model protocol

The incoming model loads:

```
AGENTS.md
DOMAIN.md
STATE.md
COMMANDS.md
DECISIONS.md
QUESTIONS.md
RISKS.md
current requirements.md
current blueprint.md
current acceptance.md
current dry_run.md
current implementation_log.md
handoff_summary.md
```

It then summarizes current state and waits for explicit mode authorization before changing files.

Rule: The next model resumes from files, not from memory.

15. Relationship to Agile and Scrum

ADDF borrows useful language from Agile. It is not Scrum.

It uses:

backlog,

sprint,

acceptance criteria,

review,

retrospective,

release,

iteration.

It does not require:

Scrum Master,

Product Owner,

daily standups,

story points,

velocity tracking,

ceremony cadence,

or two-week sprint duration.

15.1 Sprint definition

In ADDF, a sprint is a bounded AI implementation packet.

It is defined by artifacts, not by calendar duration.

15.2 Mapping

AGILE / SCRUM TERM	ADDF EQUIVALENT
Product	Project
Release	Release Cycle
Epic	Major Feature Group
User Story	Feature Brief / Requirement

AGILE / SCRUM TERM	ADDF EQUIVALENT
Sprint	AI Implementation Packet
Acceptance Criteria	`acceptance.md`
Backlog	`planning/backlog.md`
Sprint Review	Design Review + Human Review
Retrospective	`retrospective.md`
Definition of Done	`acceptance.md` + `human_review.md`
Product Owner	Human Operator
Development Team	Develop Mode + Human
Scrum Master	Not required

Rule: Use Agile language where it clarifies. Do not import Scrum ceremony.

16. Repository structure

[Image placeholder: 11. Repository Structure diagram goes here]

16.1 Standard ADDF project

```
Project-Root/
├── AGENTS.md
├── DOMAIN.md
├── STATE.md
├── DECISIONS.md
├── COMMANDS.md
├── STYLE_GUIDE.md
```

```
├── SECURITY.md
├── QUESTIONS.md
├── RISKS.md
├── GIT_STRATEGY.md
├── PROMPT_CHANGELOG.md
├── README.md
├── .gitignore
├── docs/
│   ├── architecture.md
│   ├── data_model.md
│   ├── api.md
│   ├── permissions.md
│   └── validation.md
├── research/
│   ├── notes.md
│   ├── sources.md
│   └── open_questions.md
├── planning/
│   ├── backlog.md
│   ├── releases/
│   ├── features/
│   └── sprints/
├── prompts/
│   ├── research/
│   ├── design/
│   └── develop/
└── src/
```

16.2 Minimum viable ADDF

[Image placeholder: 12. Minimal vs Full ADDF comparison diagram goes here]

```
Project-Root/
├── AGENTS.md
├── DOMAIN.md
├── STATE.md
├── COMMANDS.md
├── SECURITY.md
├── planning/
│   ├── backlog.md
│   └── sprints/
```

```
│       └── sprint_001/
│           ├── requirements.md
│           ├── blueprint.md
│           ├── acceptance.md
│           ├── dry_run.md
│           ├── implementation_log.md
│           ├── human_review.md
│           └── retrospective.md
└── src/
```

16.3 Public ADDF repository

```
autonomous-duck-deployment-framework/
├── README.md
├── START_HERE.md
├── LICENSE
├── CHANGELOG.md
├── VERSION.md
├── CONTRIBUTING.md
├── AGENTS.md
├── DOMAIN.md
├── STATE.md
├── DECISIONS.md
├── COMMANDS.md
├── STYLE_GUIDE.md
├── SECURITY.md
├── QUESTIONS.md
├── RISKS.md
├── docs/
├── starter-kit/
│   ├── blank/
│   └── example-filled/
├── prompts/
│   ├── research/
│   ├── design/
│   └── develop/
├── examples/
├── website/
├── assets/
├── brand/
```

```
├── tools/
└── planning/
```

Rule: The folder structure should make the operating model visible.

17. Initial setup

[Image placeholder: 10. First-Session Onboarding Flow diagram goes here]

17.1 Create the project

```
mkdir -p src docs research planning/sprints/sprint_001
touch AGENTS.md DOMAIN.md STATE.md DECISIONS.md COMMANDS.md STYLE_GUIDE.md SECURITY.md QUESTIONS.
touch planning/backlog.md
touch planning/sprints/sprint_001/requirements.md planning/sprints/sprint_001/blueprint.md planni
```

17.2 Populate files in order

`SECURITY.md`

`AGENTS.md`

`DOMAIN.md`

`STATE.md`

`COMMANDS.md`

`DECISIONS.md`

`QUESTIONS.md`

`RISKS.md`

17.3 First session

A new project begins with:

```
Research Mode if unknowns exist
Design Mode for project memory
Design Mode for sprint planning
Develop Mode for dry run
Develop Mode for approved implementation
Design Mode for review and state update
```

Rule: Fill `SECURITY.md` before loading files into an AI session.

18. File templates

18.1 `AGENTS.md`

```
# AGENTS.md

## Framework
This project uses the Autonomous Duck Deployment Framework.

## Central Rule
The question is never which AI does what. The question is what files must exist before any AI is

## Modes

### Research Mode
Allowed:
- Summarize sources
- Compare options
- Identify risks
- Produce research notes
```

```
- List open questions

Forbidden:
- Source code
- Final decisions
- Implementation changes

### Design Mode
Allowed:
- Create and update project brain files
- Write release plans
- Write feature briefs
- Write sprint packs
- Review dry runs
- Review implementation logs
- Write retrospectives
- Write handoff summaries

Forbidden:
- Source code
- Implementation file mutation
- Direct dependency addition

### Develop Mode
Allowed:
- Produce dry_run.md
- Modify approved files after permission
- Create tests
- Run verification commands
- Update implementation_log.md
- Produce handoff_summary.md

Forbidden:
- Skipping dry run
- Touching unapproved files
- Inventing business rules
- Adding dependencies without a decision record
- Skipping verification

## Develop Mode Permission Levels
- Level 0: Dry run only. No implementation.
- Level 1: Approved sprint scope.
- Level 2: Approved expansion.
- Level 3: Supervised refactor.
- Level 4: Migration / high-risk change.
```

```
## Session Checkpoint Rule
Before ending a session, switching models, or approaching context limits, update STATE.md and rel
```

18.2 `STATE.md`

```
# STATE.md

## Current Goal
[Current objective]

## Active Scale
[Project / Release / Feature / Sprint / Patch]

## Current Mode
[Research / Design / Develop / Human Review]

## Active Release
[Version or "None"]

## Active Feature
[Feature or "None"]

## Active Sprint
[Sprint or "None"]

## Current Status
[Research / Planning / In Progress / Verification / Complete / Blocked / Paused]

## Known Blockers
[Blockers or "None"]

## Last Completed Step
[Last completed action]

## Next Intended Step
[Next action]

## Files Recently Changed
- [file]: [summary]
```

```
## Handoff Notes
[Important continuation context]
```

18.3 DOMAIN.md

```
# DOMAIN.md

## Project Overview
[What this project does and why it exists.]

## Target Users
[Primary users]

## Core Entities

### [Entity Name]
Definition:
Key fields:
Business rules:
What it is NOT:

## Workflows

### [Workflow Name]
1. [Step]
2. [Step]

Edge cases:

## In Scope
- [Item]

## Out of Scope
- [Item]

## Non-Negotiable Rules
- [Rule]
```

18.4 DECISIONS.md

```
# DECISIONS.md

Decisions are never deleted. They are superseded by later decisions.

## Decision 001: [Short Title]
**Date:** [YYYY-MM-DD]
**Status:** Accepted / Superseded / Proposed
**Type:** Architecture / Dependency / Data Model / Security / Process / Other

### Context
[Why this decision is needed]

### Decision
[Decision made]

### Tradeoffs
[Costs, limitations, risks]

### Alternatives Considered
[Options rejected or deferred]
```

18.5 Sprint Pack

```
# requirements.md

## Sprint Goal

## Functional Requirements

## Technical Requirements

## In Scope

## Out of Scope

## Constraints
```

```
## Related Decisions

## Open Questions
```

```
# blueprint.md

## Summary

## Files to Create

## Files to Modify

## Implementation Steps

## Assumptions

## Dependencies

## Risks

## Rollback Considerations
```

```
# acceptance.md

## Functional Criteria
- [ ] [Criterion]

## Technical Criteria
- [ ] [Criterion]

## Documentation Criteria
- [ ] [Criterion]

## Verification Commands
1. [command]

## Definition of Done
- [ ] Requirements met
- [ ] Verification complete
- [ ] implementation_log.md updated
```

```
- [ ] human_review.md passed
- [ ] STATE.md updated
```

Rule: Templates are starting points, not decoration. Remove sections that do not apply.

19. Prompt catalog

Store prompts under:

```
prompts/research/
prompts/design/
prompts/develop/
```

19.1 Research Mode prompt

```
You are operating in Research Mode.

Research:
[topic]

Context:
[context]

Constraints:
[constraints]

Do not write code.
Do not make final project decisions.

Produce:
1. Summary
2. Findings
3. Options
```

```
4. Tradeoffs
5. Risks
6. Open questions
7. Recommended next step for Design Mode
```

19.2 Design Mode — project initialization prompt

```
You are operating in Design Mode for [PROJECT_NAME].

Use the provided notes, research, and constraints.

Do not write source code.

Generate:
1. DOMAIN.md
2. STATE.md
3. DECISIONS.md starter entries
4. QUESTIONS.md
5. RISKS.md
6. COMMANDS.md draft
7. STYLE_GUIDE.md draft
8. SECURITY.md draft
9. docs/architecture.md draft

Make assumptions explicit.
```

19.3 Design Mode — sprint pack prompt

```
You are operating in Design Mode.

Generate a Sprint Pack for:
[Sprint title]

Use:
- AGENTS.md
```

```
- DOMAIN.md
- STATE.md
- DECISIONS.md
- QUESTIONS.md
- RISKS.md

Do not write source code.

Output:
1. requirements.md
2. blueprint.md
3. acceptance.md

The blueprint must list exact files to create or modify.
Acceptance criteria must be checkable.
```

19.4 Develop Mode — dry run prompt

[Image placeholder: 15. CLI Initialization Flow diagram goes here]

```
You are operating in Develop Mode.

Permission Level: 0.

Read:
- AGENTS.md
- STATE.md
- COMMANDS.md
- requirements.md
- blueprint.md
- acceptance.md

Perform a read-only dry run.

Produce dry_run.md with:
1. Files to create
2. Files to modify
3. Files to move or delete
4. Commands to run
5. Dependencies requested
6. Risks
```

```
7. Ambiguities

Stop after the dry run.
Do not write code.
```

19.5 Design Mode — dry run review prompt

```
You are operating in Design Mode.

Review:
- requirements.md
- blueprint.md
- acceptance.md
- dry_run.md

Check for:
1. Scope creep
2. Missing acceptance criteria
3. Unapproved dependencies
4. Hidden refactors
5. Security issues
6. Missing rollback needs
7. Ambiguous assumptions

Return:
- Approved / Not Approved
- Required corrections
- Recommended Develop Mode permission level
```

19.6 Develop Mode — authorization prompt

```
Dry run approved.

Permission Level [LEVEL] authorized.

Proceed according to:
```

```
- requirements.md
- blueprint.md
- acceptance.md
- dry_run.md

Rules:
1. Modify only approved files.
2. Do not invent business rules.
3. Do not add dependencies unless approved.
4. Log all changes in implementation_log.md.
5. Run verification commands.
6. Paste raw output.
7. Stop if ambiguity appears.
8. Produce handoff notes before ending.
```

19.7 Resumption prompt

```
You are operating in Design Mode for resumption.

Loaded:
- AGENTS.md
- DOMAIN.md
- STATE.md
- DECISIONS.md
- QUESTIONS.md
- RISKS.md
- backlog
- current sprint or release files if present

Summarize:
1. Project purpose
2. Active scale
3. Current mode
4. Active release / feature / sprint
5. Last completed step
6. Known blockers
7. Next safest action
8. Files to load before Develop Mode
```

Rule: Prompts are executable protocol. Keep them terse.

20. Domain adaptations

ADDF is domain-agnostic. The structure remains stable; file contents adapt to the domain.

20.1 Web applications

Typical entities:

routes,

pages,

components,

API endpoints,

sessions,

database tables,

forms,

permissions.

Typical commands:

```
npm run dev
npm run test
npm run build
npm run lint
```

20.2 Game development

Typical entities:

scenes,

levels,

game states,

player states,

input actions,

assets,

UI screens,

save data.

For games, `DOMAIN.md` functions as a technical game design document.

20.3 Desktop applications

Typical entities:

windows,

menus,

local files,

system permissions,

user settings,

native services.

20.4 Data and automation

Typical entities:

input files,

output files,

rows,

transformations,

validations,

schedules,

error records.

20.5 Documentation and framework projects

Typical entities:

pages,

sections,

templates,

prompts,

examples,

downloads,

versions,

release packages.

Rule: Keep the structure stable. Adapt the domain files.

21. Worked example: Mini Task Tracker

[Image placeholder: 14. Mini Task Tracker Scale Example diagram goes here]

21.1 Work scale

```
Project: Mini Task Tracker
Release: v0.1 Local MVP
Feature: Core task list
Sprint: Sprint 001 — Add, display, complete, and persist tasks
```

21.2 Lifecycle use

Research Mode:

```
Investigate localStorage and client-only task persistence.
```

Design Mode:

```
Create DOMAIN.md, STATE.md, DECISIONS.md, requirements.md, blueprint.md, and acceptance.md.
```

Develop Mode:

```
Dry run, implement TaskForm, TaskList, storage helper, tests, and implementation_log.md.
```

Design Mode:

```
Review implementation output, update STATE.md, and record retrospective notes.
```

21.3 Example `DOMAIN.md`

```
# DOMAIN.md

## Project Overview
Mini Task Tracker is a local-only task management app for a single user.

## Core Entities

### Task
Definition: A single item the user wants to complete.
Key fields: id, title, status, createdAt, completedAt.
Business rules:
- A task title cannot be empty.
- A completed task can be reopened.
- Tasks are stored in browser localStorage.
What it is NOT:
- A shared task
```

```
- A cloud record
- A calendar event
- A reminder

## Workflows

### Add a Task
1. User types a title.
2. User submits.
3. Task is created with active status.
4. Task is saved to localStorage.

Edge case:
If the title is empty or whitespace-only, reject submission.

## Out of Scope
- User accounts
- Cloud sync
- Notifications
- Collaboration
```

Rule: Small examples stay small. The point is to expose the process, not showcase an app.

22. Worked example: ADDF public repository

[Image placeholder: 13. ADDF Public Repository Roadmap diagram goes here]

22.1 Project scope

The public ADDF repository includes:

documentation,

website,

starter kit,

prompt catalog,

example project,

download packages,

optional CLI init tool,

optional web onboarding app.

22.2 Release plan

v0.1 Public Proof

```
README.md
START_HERE.md
Full manual Markdown
Full manual PDF
starter-kit/blank
starter-kit/example-filled
examples/mini-task-tracker
prompts/
assets/logo
assets/lifecycle image
```

v0.2 Website MVP

```
Static website
Homepage
Start Here page
Lifecycle page
Modes page
Downloads page
```

v0.3 Init Tool Prototype

```
addf init
minimal/full setup
project-type presets
generated starter files
```

v0.4 Web Onboarding App

```
Browser questionnaire
Generated file preview
Download starter ZIP
Copy prompt buttons
```

22.3 Sprint 001

Goal:

```
Normalize repository structure and starter kit so the repo becomes the official ADDF project home
```

In scope:

organize docs,
create starter-kit folders,
create prompt folders,
create planning structure,
create example placeholders,
create `README.md` and `START_HERE.md` if missing.

Out of scope:

website implementation,
CLI implementation,
backend,
package publishing,

final branding.

Rule: The public repository must dogfood the framework.

---

23. Operational checklists

23.1 Before first AI session

- [ ] `SECURITY.md` exists.
- [ ] Never-share items are defined.
- [ ] Work scale is identified.
- [ ] Project goal is written.
- [ ] Research Mode is used if unknowns exist.
- [ ] Design Mode is used before Develop Mode.

23.2 Before Develop Mode

- [ ] Sprint Pack exists.
- [ ] `requirements.md` is clear.
- [ ] `blueprint.md` lists files.
- [ ] `acceptance.md` is checkable.
- [ ] Branch is created.
- [ ] Permission Level 0 is declared.
- [ ] Develop Mode is instructed to dry run and stop.

23.3 Before authorizing development

- [ ] `dry_run.md` matches `blueprint.md`.

- [ ] Design Mode review passed.
- [ ] Dependencies are approved.
- [ ] Risks are acceptable.
- [ ] Rollback exists if needed.
- [ ] Permission level is explicit.

23.4 Before commit

- [ ] `implementation_log.md` is complete.
- [ ] Verification output is pasted.
- [ ] `human_review.md` passed.
- [ ] `DECISIONS.md` updated.
- [ ] Backlog updated.
- [ ] `STATE.md` updated.
- [ ] `retrospective.md` completed.
- [ ] Handoff notes exist if pausing.

Rule: A checklist is an execution gate, not a suggestion.

24. Failure handling

24.1 Develop Mode touched unapproved files

Stop implementation.

Actions:

Record the issue in `implementation_log.md`.

Compare changes against `dry_run.md`.

Revert unapproved changes if needed.

Update `RISKS.md`.

Add a constraint to `AGENTS.md`.

24.2 The model invented business logic

Actions:

Identify the invented logic.

Check `DOMAIN.md`.

Remove the logic or mark it for review.

Update `DOMAIN.md` only if the logic is approved.

Record the issue in `retrospective.md`.

Add a Develop Mode constraint.

24.3 Project state is confused

Use Design Mode for a consistency audit.

Load:

```
DOMAIN.md
STATE.md
DECISIONS.md
recent sprint folders
docs/architecture.md
```

Ask Design Mode to identify drift and recommend updates.

24.4 Model runs out of context

Use Session Checkpoint Protocol.

Require:

```
STATE.md update
implementation_log.md update if applicable
handoff_summary.md
```

Open a new session and run the Resumption Prompt.

Rule: Failure recovery begins by writing state, not by asking for another fix.

25. Glossary

TERM	DEFINITION
ADDF	Autonomous Duck Deployment Framework.
Context Engineering	Organizing project context into explicit, scoped, durable files.
Design Mode	The operating mode that writes Markdown project memory.
Develop Mode	The operating mode that writes code or implementation files after approval.
Dry Run	A Develop Mode report describing intended changes before implementation.
Feature	A capability added to an existing project.
Handoff Summary	A written continuation record for another model or human.
Local File Sovereignty	The principle that project memory lives in local files, not model memory.
Patch	A tiny low-risk change.
Project brain	The collection of Markdown files that define current project truth.
Release	A versioned delivery target.

TERM	DEFINITION
Research Mode	The operating mode that investigates and synthesizes information.
Sprint	A bounded AI implementation packet.
Sprint Pack	`requirements.md`, `blueprint.md`, and `acceptance.md`.
Validation Gate	A decision checkpoint before architecture, sprint planning, or implementation.

Closing principle

ADDF is a coordination framework, not a coding trick.

Use Research Mode to establish evidence.

Use Design Mode to make project memory explicit.

Use Develop Mode to modify implementation under reviewable constraints.

Choose the scale before choosing the process.

Run only as much lifecycle as the work requires.

Keep the source of truth in files.

Let the model assist. Do not let the model become the memory of the project.

Quack!